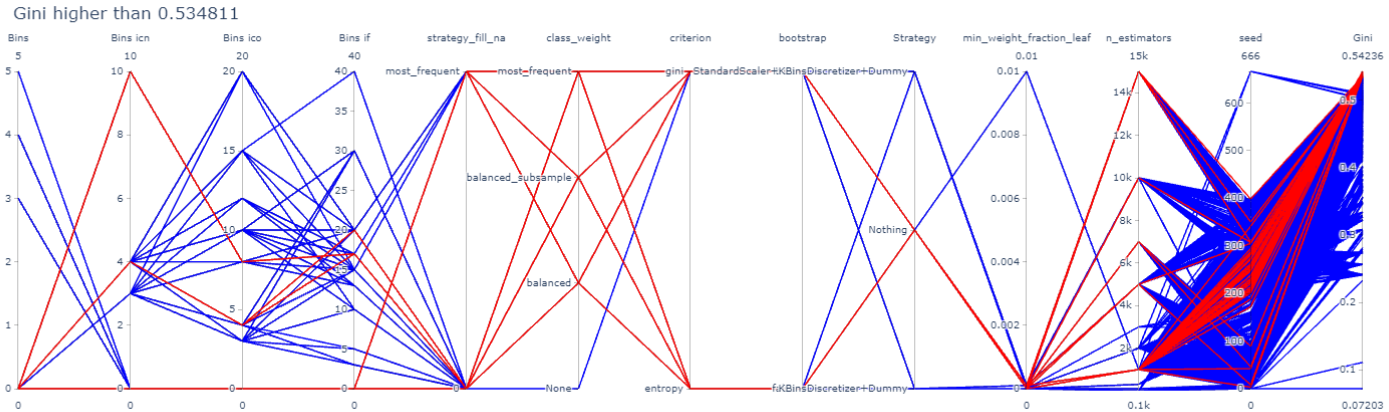
Juan Pedro Bretti Mandarano
Group: BLENDED
Email: juanbretti@student.ie.edu
User: juanbretti
Best Gini: 0.58248
Stand Dev: 0.07407232306782156
Total Submissions: 5025



Tried and provides improvements

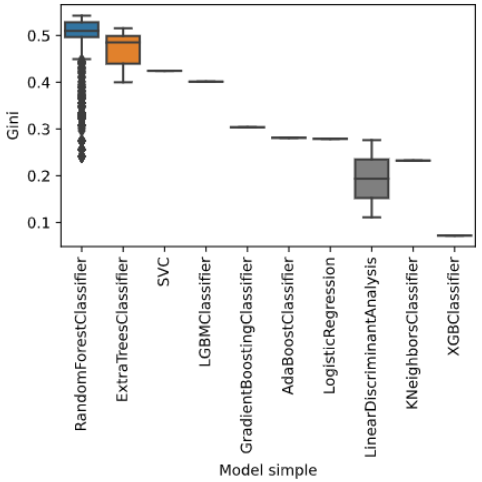
- The model use was Random Forest Classifier (RFC)
  - This algorithm performs best for this binary classification, because its ability to capture non-linear relationships. Also, after being an ensemble method (train multiple models instead of a single model and to combine the predictions from these models).
  - Random variation of the *random_state* or *seed*. Helped because this parameter defines the starting point, and also could affect the number of frauds on each partition.
  - Most of the iterations was done
    - *bootstrap=False*, to reduce the variance of the training dataset. The whole dataset is used to build each tree
    - *class_weight = 'balanced_subsample'*. Helped because weights are computed based on the bootstrap sample for every tree grown. This is necessary to manage the unbalanced dataset. Mostly, considering I was disabled SMOTE or any other alternative.
- Fill missing values with *most frequent* or *0*. This preprocessing is necessary, but significant difference between the two options.
- Ensure all the points in the *training* set were having a perfect *accuracy* in the *test* set (all the training points has their match in the test set)
- Genetic algorithm (GA) for feature selection (after professor's code). Some alterations were done.
  - GA is an optimization method, in this case was used to reduce the number of variables
  - The feature selection using GA helped by reducing bias and variance captured by the model *fit()*, affecting the generalization.
  - Sensitivity to number of *n_estimators* to improve performance.
- Most of the previously listed sensitivities, and some more, are in the top right **plot** (done before the GA applied).
  - Notice, to achieve a, e.g. Gini > 0.5348, there are multiple paths, and it's noticeable there are some seeds that have more probability (close to 200).
  - This is expected, considering the data split generated by the multiple *seeds*, and the low number of frauds in the dataset.

Pending ideas to test

- Grid search between RFC parameters, GA, and seed, using the server for scoring
- Feature selection using SequentialFeatureSelector, using the server for scoring
- Model ensemble using BaggingClassifier or Voting Classifier

Did not worked

- Train test split was not tested, expecting decrease in performance. Because, on each split the number of frauds was reduced and the representativity.
- Multiple models, with multiple parameters. **Plot** at the bottom right. Models like XGBoost are very powerful to be trained, but hard to tune to generalize considering the low number of fraud datra points.
- Multiple variations of SMOTE for oversampling. Failed compared to the *class_weight* setting.
- Feature's preprocessing (multiple strategies were tested):
  - Binning the features with multiple number of K, using KBinsDiscretizer. Failed because generalized too much.
  - Scaling. Was not necessary considering the predominant model –RFC- does not benefit from normalization.
  - PCA for feature space reduction. Failed because of too much generalization.
- Custom metric, using "Cost" based on class documentation. This function needs to be tuned.
- Features selection using:
  - Variable importance (from RFC). The method was not compared to the *test results* at the server.
  - IV, WoE and PSI (from professor's code) . The method was not compared to the *test results* at the server.
  - Using SequentialFeatureSelector, with *training* split (too slow to run). For better performance, must be tested to the server.
  - Genetic algorithm, tested with *training* Split.
- Changes in the *criterion* for RFC. Not relevant improvement.
- The alternative server had an issue for Gini > 0.56024



Files included

- 6aQ6IxU7Va_grid_search.py: Grid search
- SMOTE+Cost.py: Test of multiple ideas
- GA_feature_selection_5_manoelutad.py: GA using the best model from previous
- 6aQ6IxU7Va_GA_1.py: After the GA, small additional grid search
- df_test_tosend_GA_1_manoelutad.csv: CSV used to archieve the highest score
- Merge store plot.py: Plots in this page